

DEEP LEARNING

GROUP ALPHA FINAL PROJECT

BASICS ON CONVOLUTIONAL NEURAL NETWORKS AND DATA LOADING IN PYTORCH

Albert Cirera Lacasa, Pere Caballé Jiménez, Guillem González Casanova

1. Introducción:

La detección de puntos faciales consiste en localizar 68 puntos anatómicamente significativos en una imagen de cara humana (comisuras de los ojos, punta de la nariz, extremos de la boca, contorno de la mandíbula) como base para seguimiento facial, análisis de expresiones y reconocimiento. Desde el punto de vista del deep learning, se trata de un problema de regresión: el modelo predice un vector de 136 valores continuos (x,y por keypoint) usando MSE (L2-loss) como función de pérdida: $L(\hat{y}, y) = \|\hat{y} - y\|^2$, que penaliza cuadráticamente los errores grandes, permitiendo el entrenamiento mediante backpropagation. La motivación es explorar el uso de CNN para regresión en imágenes y evaluar la capacidad de generalización sobre datos nunca vistos.

2. Estado del Arte:

Los métodos clásicos como ASM y AAM ajustan iterativamente los puntos mediante restricciones estadísticas. Las CNN supera estas limitaciones usando la estructura composicional de las imágenes: features locales, estacionarios y multi-escala, con compartición de parámetros que reduce dramáticamente el coste respecto a redes fully connected.

La competición Kaggle Facial Keypoints Detection [2] estableció la regresión directa de coordenadas como enfoque estándar. NaimishNet (Agarwal et al., 2017) [1], arquitectura de referencia de dicha competición, añade Batch Normalization, activación ELU y Dropout progresivo.

Este trabajo aplica regresión directa con CNN al dataset YouTube Faces [3] mayor escala y variabilidad que el benchmark de Kaggle con 68 keypoints en RGB (136 salidas) frente a 15 en escala de grises (30 salidas). La contribución principal es la comparativa entre seis configuraciones de arquitectura, datos y learning rate, evaluadas sobre datos completamente no vistos.

Como séptimo experimento, se evalúa la implementación de QAT (Quantization Aware Training) para usar la CNN en dispositivos con recursos computacionales limitados, con el objetivo de comprimir la memoria y acelerar la velocidad de inferencia, sin perder precisión.

3. Metodología

3.1 Análisis del dataset

Dataset: YouTube Faces with Facial Keypoints (Kaggle). 260.000 frames válidos en 1.245 archivos .npz. Cada archivo almacena colorImages (H,W,3,frames) uint8 y landmarks2D (68,2,frames) float64. División: 80% train 20% validación de una muestra aleatoria (seed=42), más un conjunto de test de 10.000 frames del pool de 210.000 frames nunca usados en entrenamiento ni validación.

Preprocesamiento: Primero Redimensionamiento a 96×96 px bilineal, segundo normalización de píxeles $\div 255 \rightarrow [0,1]$, tercero transposición (H,W,C) $\rightarrow$ (C,H,W) para PyTorch, cuarto normalización de keypoints $x \div w, y \div h \rightarrow [0,1]$, quinto eliminación de frames con NaN. Los arrays preprocesados se

guardan como .npy para evitar lecturas repetidas de NPZs en cada época clave para el rendimiento en CPU/DirectML. Con batch size 64 y 40.000 frames de entrenamiento, cada época implica 625 iteraciones.

3.2 Modelos y optimización

Baseline CNN 4 bloques convolucionales (Conv2d→ReLU→MaxPool2d) reduciendo la resolución espacial $96 \rightarrow 48 \rightarrow 24 \rightarrow 12 \rightarrow 6$ y los canales $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$, seguidos de FC($9216 \rightarrow 1024 \rightarrow 256 \rightarrow 136$)+Sigmoid. ReLU evita el vanishing gradient. Sigmoid fuerza la salida a $[0,1]$ consistente con los keypoints normalizados. 10,1M parámetros entrenables.

También, sobre la Baseline se evalúa el efecto de la data augmentation [5], aplicada on-the-fly solo en entrenamiento (validación y test intactos). Incluye transformaciones geométricas (flip horizontal) con remapeo de keypoints, rotación ($\pm 12^\circ$) y desplazamiento (± 6 px), aplicadas también a las coordenadas, jitter de brillo y ruido gaussiano, que no alteran los keypoints.

Adicionalmente, para saber si se puede comprimir el modelo de Baseline CNN, se modificó su estructura utilizando el framework `torch.quantization` [6]. El proceso incluye: inclusión de los módulos `QuantStub` y `DeQuantStub` al inicio y final de la red, la fusión de los bloques Conv2d + ReLU y Linear + Relu para reducir los accesos a la memoria, el entrenamiento en float32 pero simulado a int8 i finalmente se convierte mediante `torch.quantization.convert` a operaciones de enteros puros de 8 bits (int8).

NaimishNet (Agarwal et al., 2017) [1] misma estructura con tres mejoras de regularización: primero Batch Normalization tras cada conv (normaliza activaciones por mini-batch), segundo ELU ($f(x)=x$ si $x \geq 0$, $\alpha(e^x-1)$ si $x < 0$ evita neuronas muertas, centra activaciones en torno a cero), tercero el dropout progresivo ($p=0,1 \rightarrow 0,4$ en bloques conv, $p=0,5$ en FC). Sin Sigmoid regresión directa sin restricción de rango. Kernels decrecientes 4,3,2,1.

Entrenamiento: Optimizador Adam (learning rate adaptativo mediante primero y segundo momento del gradiente), scheduler ReduceLROnPlateau (factor=0,5, patience=3), MSELoss, 20 epochs, mini-batch 64. Baseline $lr=1e-3$, NaimishNet probado con $1e-3$ y $1e-4$.

4. Experimentos

Cinco experimentos aíslan tres factores independientes, arquitectura, volumen de datos y learning rate:

- Exp. 1: Baseline CNN — 50k frames, $lr=1e-3$ (referencia)
- Exp. 2: Baseline CNN + Data Augmentation — 50k frames, $lr=1e-3$ (transformaciones)
- Exp. 3: NaimishNet — 50k frames, $lr=1e-3$ (efecto de arquitectura)
- Exp. 4: NaimishNet — 100k frames, $lr=1e-3$ (efecto de datos)
- Exp. 5: NaimishNet — 50k frames, $lr=1e-4$ (efecto del learning rate)
- Exp. 6: NaimishNet — 100k frames, $lr=1e-4$ (todas las mejoras combinadas)
- Exp. 7: Baseline CNN + QAT t — 50k frames, $lr=1e-4$

Todos los modelos se entrenan monitorizando curvas de train/validation loss para detectar overfitting. El checkpoint guardado corresponde a la época con menor validation loss. Adicionalmente se realiza un análisis de errores sobre 10.000 frames de test para identificar patrones de fallo sistemáticos.

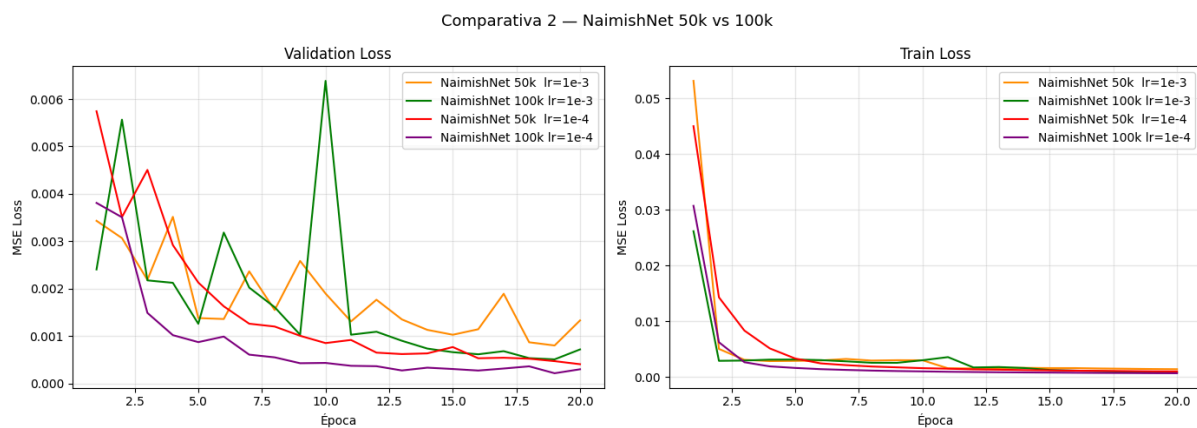
5. Resultados

5.1 Comparativa validación

Modelo	Frames	LR	Mejor val MSE	RMSE (px)
Baseline CNN	50k	1e-3	0,000065	0,89 px
Baseline + Augmentation	50k	1e-3	0,000084	1,05 px
Baseline + QAT	50k	1e-4	0.000093	1.07 px
NaimishNet	50k	1e-3	0,000802	3,15 px
NaimishNet	100k	1e-3	0,000510	2,36 px
NaimishNet	50k	1e-4	0,000408	2,26 px
NaimishNet	100k	1e-4	0,000217	1,64 px

La Baseline CNN obtiene el mejor RMSE (0,89 px). La data augmentation (Exp. 2) no mejora el RMSE de validación (0,89 $\rightarrow$ 1,05 px), resultado esperable al no haber overfitting en la Baseline. Por otro lado la Baseline CNN con QAT un buen rendimiento, alcanzando un MSE mínimo de validación de 0.000093 y un RMSE global de 1.07 px, pero no supera a la Baseline básica .

NaimishNet con lr=1e-3 muestra convergencia inestable consistente con la teoría: un learning rate demasiado alto provoca que el optimizador sobrepase los mínimos, visible como picos en la loss alrededor de la época 10. Reducir a lr=1e-4 estabiliza el entrenamiento y mejora el RMSE de 3,15 $\rightarrow$ 2,26 px (50k) y 2,36 $\rightarrow$ 1,64 px (100k), pero en ninguna configuración supera a la Baseline.



5.2 Generalización test con datos no vistos

Conjunto	Frames	RMSE Baseline	RMSE NaimishNet
Validación	10.000	0,89 px	1,64 px
Test (no vistos)	10.000	0,90 px	1,63 px
Diferencia	—	0,1%	0,6%

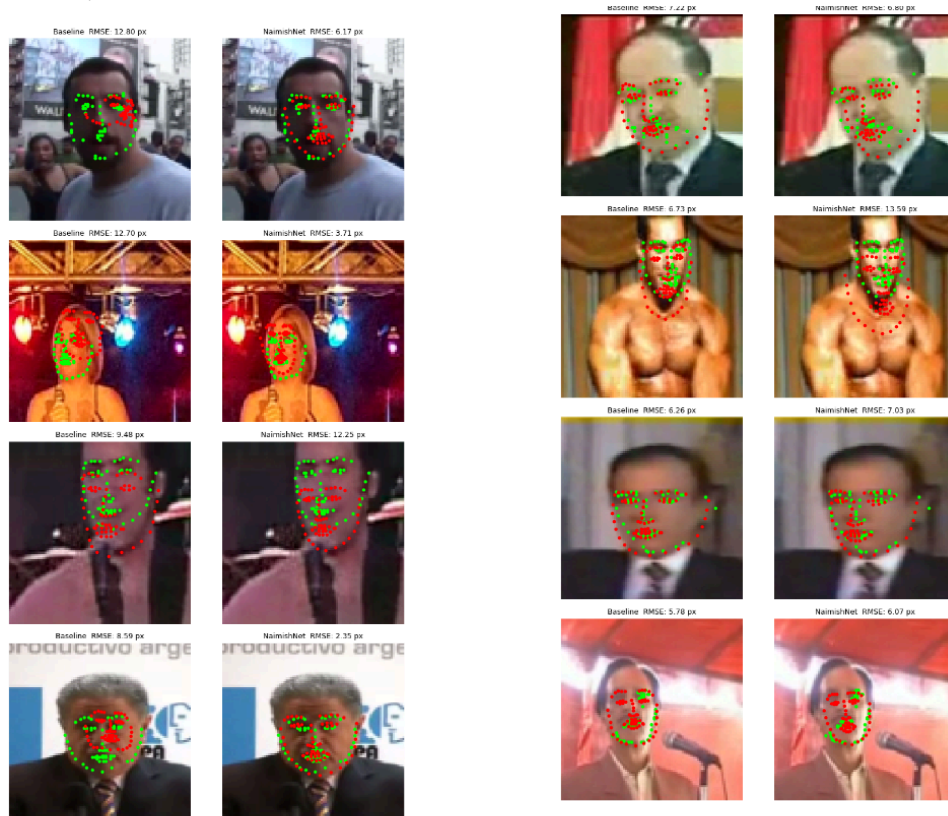
Ambos modelos generalizan correctamente. La diferencia val, test es 0,1% para Baseline y 0,6% para NaimishNet, confirmando ausencia de overfitting.

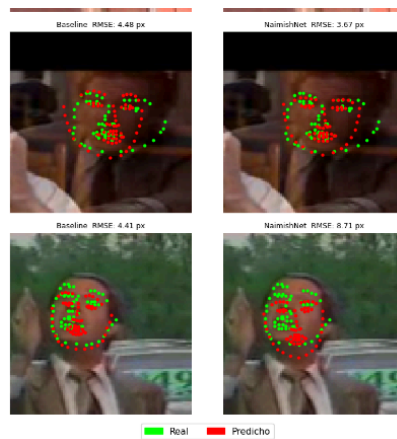
5.3 Análisis de errores

Percentil	Baseline CNN (px)	Baseline + Augmentation	Baseline + QAT	NaimishNet (px)	Dif. (Naimish - Baseline)
p50	0,80	1,115	1.10 px	1,49	+0,69
p90	1,21	1,570	1.65 px	2,32	+1,11
p99	1,95	2,167	2.91 px	3,97	+2,02
Media	0,89	1,05	1.21 px	1,63	+0,74
Frames >5 px	8 (0,1%)	6 (0,06%)	11 (0,11%)	36 (0,4%)	4×

La Baseline es mejor en todos los percentiles. Frames con RMSE > 5 px: 8 (0,1%) para Baseline, 6 (0,06%) para Baseline con augmentation, 11 (0,11%) para Baseline con QAT y 36 (0,4%) para NaimishNet, 4 veces más casos extremos respecto al baseline. La inspección visual de los 20 peores frames en el caso de Naimish muestra cuatro patrones de fallo: color (cara y ropa tienen la misma tonalidad de color), iluminación extrema (focos de colores, contraluz), cara de perfil (giro >45°) y texto. La comparativa directa muestra que cada modelo falla en frames distintos.

Top 10 frames difíciles — Baseline vs NaimishNet



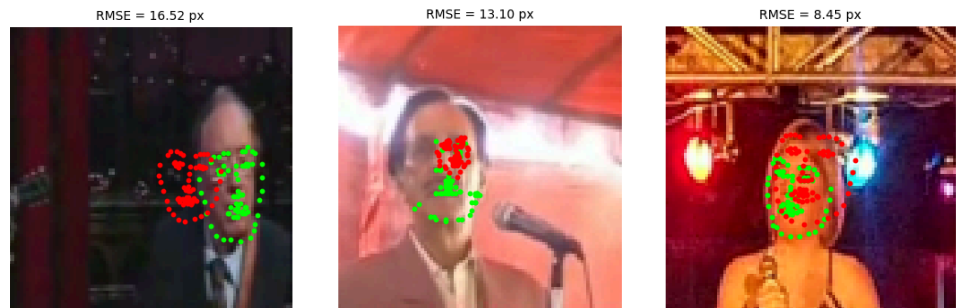


6. Discusión

6.1 Data Augmentation

El análisis de la distribución de errores de la Baseline con augmentation matiza el resultado de la media. Aunque los percentiles centrales empeoran ligeramente (p50: 0,80→1,12 px; p90: 1,21→1,57 px) y el p99 sube de forma moderada (1,95→2,16 px), la cola extrema se acorta: los frames con error mayor de 5 px se reducen de 8 (0,1%) a 6 (0,06%). Es decir, la augmentation no mejora el caso típico pero reduce los fallos catastróficos, justo en los frames difíciles (oclusión, iluminación, perfil) identificados como puntos débiles del modelo.

BaselineCNN



BaselineCNN + Augmentation



6.2 QAT

Los resultados del Exp. 7 validan el uso de QAT como una estrategia de optimización excelente. Una conversión directa tradicional (PTQ) habría roto la continuidad de los mapas de características necesarios para la regresión precisa de keypoints debido al severo truncamiento numérico. Sin embargo, mediante QAT, la red aprende a ser resiliente a esta restricción. Desde una perspectiva de ingeniería de software, sustituir el almacenamiento y cómputo FP32 por INT8 reduce teóricamente el tamaño del archivo de pesos, además de multiplicar la velocidad de ejecución y disminuir el consumo energético en plataformas de destino CPU. Un aumento de error insignificante de poco más de 0.3 píxeles es un coste totalmente asumible a cambio de lograr un modelo de producción preparado para tiempo real y entornos embebidos.

6.3 NaimishNet

El resultado más notable es que la Baseline CNN supera a NaimishNet en todas las configuraciones. Una explicación puede ser la regularización excesiva: NaimishNet fue diseñado para un dataset distinto (escala de grises, 15 keypoints) y su Dropout (0,5 en FC) introduce demasiado ruido para el volumen de datos y la tarea actuales, impidiendo que la red aprenda representaciones útiles. Además, la interacción Batch Normalization más Dropout en capas densas puede ser inestable.

El efecto del learning rate es más común: con $lr=1e-3$ el gradient descent oscila alrededor del mínimo (picos en las curvas), con $lr=1e-4$ converge establemente. El scheduler ReduceLROnPlateau refuerza este efecto reduciendo el LR cuando la validation loss se estanca.

6.4 Limitaciones y trabajo a futuro

El estudio presenta varias limitaciones: la data augmentation y QAT solo se evaluó sobre la Baseline y no sobre NaimishNet, el dataset se limita a caras de celebridades (posible sesgo) y los hiperparámetros de NaimishNet no se ajustaron para esta tarea.

La línea más inmediata es aplicar data augmentation a NaimishNet: como sobre la Baseline ya ha demostrado aportar robustez frente al peor caso, resulta prometedor combinarla con un modelo más regularizado. Además, se plantea evaluarla sobre múltiples seeds y con políticas automáticas (AutoAugment, RandAugment), probar augmentation de forma offline en lugar de on-the-fly para añadir más variabilidad partiendo de los mismos frames originales, explorar transfer learning con ResNet50 y ajustar el Dropout de NaimishNet o usar un scheduler con warm-up.

7. Conclusiones

La Baseline CNN alcanza un RMSE de 0,89 px (validación) / 0,90 px (test), precisión subpíxel con una diferencia de solo 0,1%, lo que confirma la ausencia de overfitting. La función MSE (L2-loss) resulta adecuada para la regresión de coordenadas: penaliza cuadráticamente los errores grandes y es diferenciable, permitiendo backpropagation y gradient descent.

El learning rate es crítico: NaimishNet mejoró 4× (3,15→1,64 px) solo reduciendo el lr de 1e-3 a 1e-4, ya que un valor demasiado alto provoca oscilaciones en el gradient descent. Asimismo, la regularización debe ajustarse al dataset: la configuración de NaimishNet, pensada para otra tarea, penaliza su rendimiento en este problema.

Respecto a la data augmentation, su efecto sobre la Baseline confirma que en un modelo que ya generaliza sin overfitting no mejora la precisión media (0,89→1,05 px), pero sí aporta robustez frente al peor caso: reduce los fallos catastróficos en los frames difíciles (oclusión, iluminación extrema, perfil). Es decir, intercambia algo de precisión típica por una menor tasa de errores extremos, lo que la hace especialmente prometedora para modelos más sensibles a estos casos.

Finalmente, la integración exitosa de Quantization Aware Training (QAT) demuestra que es viable comprimir drásticamente la huella de memoria y acelerar los tiempos de cálculo en modelos convolucionales de regresión espacial. Esto permite ejecutar programas en dispositivos menos potentes sin llegar a perder mucha precisión.

8. Referencias

[1] mminamina. (2025). GitHub - mminamina/Transfer_Learning_NaimishNet---Keypoints_Detection: Keypoints Detection with Transfer Learning(NaimishNet) in Pytorch. GitHub.
https://github.com/mminamina/Transfer_Learning_NaimishNet---Keypoints_Detection

[2]Facial Keypoints Detection. (2026). @Kaggle.
<https://www.kaggle.com/competitions/facial-keypoints-detection>

[3] Beniaguev, D. (2017). YouTube Faces With Facial Keypoints. Kaggle.com.
<https://www.kaggle.com/datasets/selfishgene/youtube-faces-with-facial-keypoints>

[4] ishandutta0098. (2025). GitHub - ishandutta0098/facial-keypoints-detection: Deep Learning Models for Facial Keypoints Detection. GitHub.
<https://github.com/ishandutta0098/facial-keypoints-detection>

[5] balraj98. (2020, December). Data Augmentation for Facial Keypoint Detection. Kaggle.com; Kaggle. <https://www.kaggle.com/code/balraj98/data-augmentation-for-facial-keypoint-detection>

[6] Service Not Allowed. (2024). Google.com.
https://colab.research.google.com/github/ishandutta0098/facial-keypoints-detection/blob/main/notebooks/quantization_aware_training.ipynb#scrollTo=13SubKug4efe